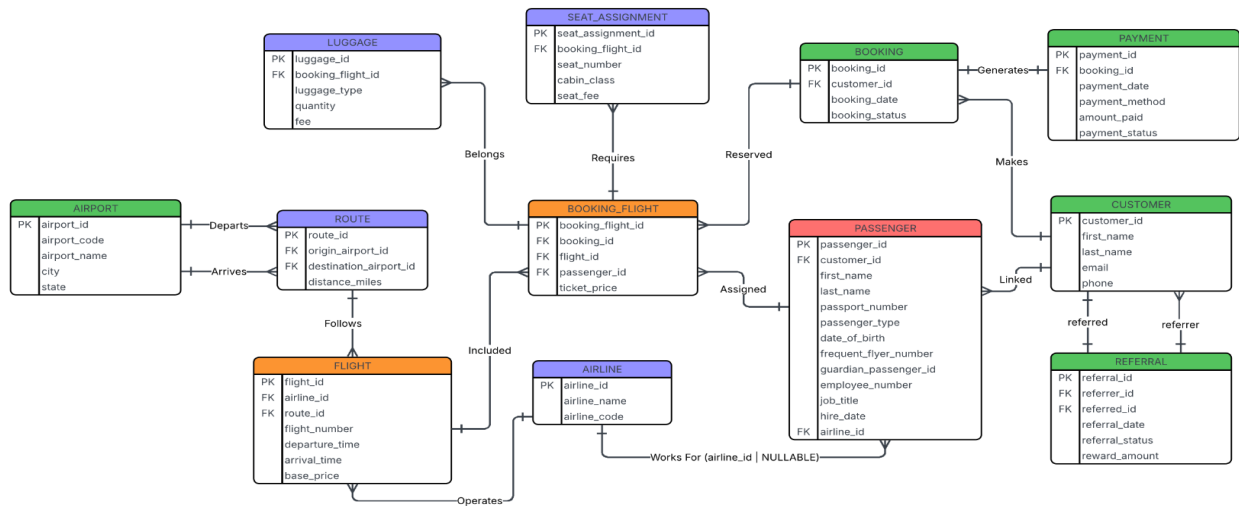


BUS 393
FINAL WRITE UP PART B
AIRLINE BOOKING PLATFORM

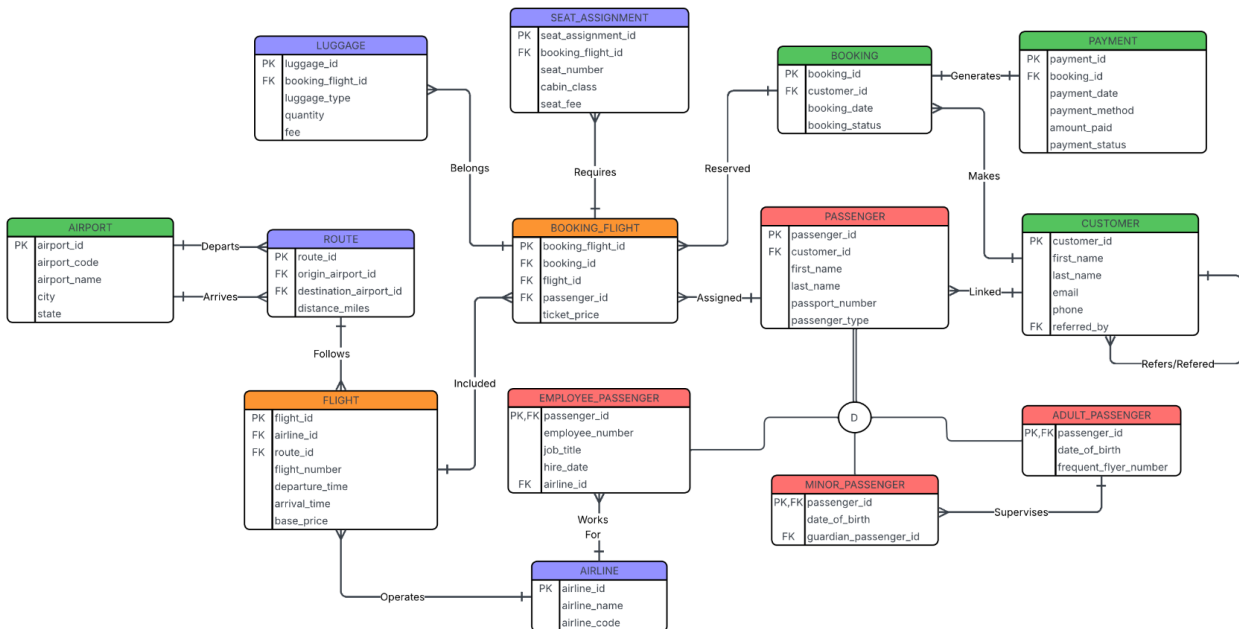
Team composition: Christina Balingit, Maya V. Karl, Marshall Moore, Arturo Ordaz-Gutiérrez, Courtney Yuen

Normalization:

Original ERD:



Updated ERD:



Change 1 referral removed: The original ERD had a referral entity with referral_date, referral_status, and reward_amount. Those attributes assumed the platform would manage referral rewards. We scoped that out because referral programs are run by the airlines themselves, not the booking platform. With no attributes of its own, the entity had no reason to exist and collapsed into a single referred_by FK on customer.

Change 2 passenger table supertype: The original ERD had one flat passenger table with 7 type-specific columns that were NULL for most rows. Nothing enforced that employees had airline_id or that minors had guardian_passenger_id, that was left to application code. The disjoint hierarchy fixes this by giving each type its own table where every column is always required and always meaningful.

De-normalization: No de-normalization was performed. All tables are in third normal form: every non-key attribute depends on the whole primary key, and there are no transitive dependencies. We checked the functional dependencies after transforming the ER model and found no partial or transitive dependencies that required splitting or merging tables.

Create Database: The following CREATE statements define all 14 tables, including primary keys, foreign keys, the disjoint passenger hierarchy, and the CHECK constraint on passenger_type. Tables are created in foreign-key dependency order.

```
CREATE TABLE customer (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    gmail VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(20),  
    referred_by INT DEFAULT NULL,  
    FOREIGN KEY (referred_by) REFERENCES customer(customer_id)  
);
```

```
CREATE TABLE airline (  
    airline_id INT PRIMARY KEY AUTO_INCREMENT,  
    airline_name VARCHAR(100) NOT NULL,  
    airline_code VARCHAR(10) NOT NULL UNIQUE  
);
```

```
CREATE TABLE airport (  
    airport_id INT PRIMARY KEY AUTO_INCREMENT,  
    airport_code VARCHAR(10) NOT NULL UNIQUE,  
    airport_name VARCHAR(100) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    state VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE route (  
    route_id INT PRIMARY KEY AUTO_INCREMENT,  
    origin_airport_id INT NOT NULL,  
    destination_airport_id INT NOT NULL,
```

```

        distance_miles INT NOT NULL,
        FOREIGN KEY (origin_airport_id) REFERENCES airport(airport_id),
        FOREIGN KEY (destination_airport_id) REFERENCES airport(airport_id),
        CHECK (origin_airport_id <> destination_airport_id)
    );

CREATE TABLE flight (
    flight_id INT PRIMARY KEY AUTO_INCREMENT,
    airline_id INT NOT NULL,
    route_id INT NOT NULL,
    flight_number VARCHAR(20) NOT NULL,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL,
    base_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (airline_id) REFERENCES airline(airline_id),
    FOREIGN KEY (route_id) REFERENCES route(route_id)
);

CREATE TABLE booking (
    booking_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    booking_date DATE NOT NULL,
    booking_status VARCHAR(20) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

CREATE TABLE passenger (
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT DEFAULT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    passport_number VARCHAR(30),
    passenger_type VARCHAR(10) NOT NULL CHECK (passenger_type IN ('adult', 'minor', 'employee')),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

CREATE TABLE adult_passenger (
    passenger_id INT PRIMARY KEY,
    date_of_birth DATE NOT NULL,
    frequent_flyer_number VARCHAR(30),
    FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id)
);

CREATE TABLE minor_passenger (
    passenger_id INT PRIMARY KEY,
    date_of_birth DATE NOT NULL,
    guardian_passenger_id INT NOT NULL,
    FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id),
    FOREIGN KEY (guardian_passenger_id) REFERENCES adult_passenger(passenger_id)
);

CREATE TABLE employee_passenger (
    passenger_id INT PRIMARY KEY,

```

```

        employee_number VARCHAR(20) NOT NULL UNIQUE,
        job_title VARCHAR(50) NOT NULL,
        hire_date DATE NOT NULL,
        airline_id INT NOT NULL,
        FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id),
        FOREIGN KEY (airline_id) REFERENCES airline(airline_id)
    );

CREATE TABLE booking_flight (
    booking_flight_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_id INT NOT NULL,
    flight_id INT NOT NULL,
    passenger_id INT NOT NULL,
    ticket_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (booking_id) REFERENCES booking(booking_id),
    FOREIGN KEY (flight_id) REFERENCES flight(flight_id),
    FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id)
);

CREATE TABLE payment (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_id INT NOT NULL,
    payment_date DATE NOT NULL,
    payment_method VARCHAR(30) NOT NULL,
    amount_paid DECIMAL(10,2) NOT NULL,
    payment_status VARCHAR(20) NOT NULL,
    FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
);

CREATE TABLE luggage (
    luggage_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_flight_id INT NOT NULL,
    luggage_type VARCHAR(30) NOT NULL,
    quantity INT NOT NULL,
    fee DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (booking_flight_id) REFERENCES booking_flight(booking_flight_id)
);

CREATE TABLE seat_assignment (
    seat_assignment_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_flight_id INT NOT NULL,
    seat_number VARCHAR(10) NOT NULL,
    cabin_class VARCHAR(30) NOT NULL,
    seat_fee DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (booking_flight_id) REFERENCES booking_flight(booking_flight_id)
);

```

Populate: The database is populated from 14 CSV files, one per table, loaded by a Python script.

Data was populated using a Python script (load_data.py) that reads 14 CSV files, one per table, and loads them into the database. For each row, the script builds and executes a parameterized

INSERT statement, so the effect is the same as running individual SQL INSERT statements, but the values are supplied safely from the CSV files. INSERT IGNORE is used so the script can be re-run without raising duplicate-key errors, and any text value of “NULL” in a CSV is converted to a real SQL NULL to support the nullable foreign keys (referred_by and customer_id). The files are loaded in foreign-key dependency order, listed in LOAD_ORDER below, so that every referenced row exists before it is referenced.

The row counts produced by the loader are listed below. Every table holds at least 10 records except the minor_passenger subtype. This is expected: the three passenger subtype tables (adult_passenger, minor_passenger, and employee_passenger) together partition the 25-row passenger supertype, so each passenger appears in exactly one subtype table and their counts sum to the passenger total rather than each table separately reaching 10.

Row counts:

```
customer: 1000
airline: 10
airport: 20
route: 50
flight: 300
booking: 1000
passenger: 25
adult_passenger: 12
minor_passenger: 3
employee_passenger: 10
booking_flight: 1000
payment: 1000
luggage: 1000
seat_assignment: 1000
```

```
import csv
import mysql.connector
import os
import re

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "YOUR_ROOT_PASSWORD_HERE",
}

DATABASE = "airline_booking"
SQL_FILE = os.path.join(os.path.dirname(__file__), "airline_booking.sql")
DATA_DIR = os.path.join(os.path.dirname(__file__), "..", "Data")

# CSV filename -> table name (only where they differ)
TABLE_MAP = {"seat_assignments.csv": "seat_assignment"}

# CSV column -> table column (only where they differ)
COLUMN_MAP = {"customer.csv": {"email": "gmail"}}

# Order respects FK dependencies
LOAD_ORDER = [
```

```

"customer.csv",
"airline.csv",
"airport.csv",
"route.csv",
"flight.csv",
"booking.csv",
"passenger.csv",
"adult_passenger.csv",
"minor_passenger.csv",
"employee_passenger.csv",
"booking_flight.csv",
"payment.csv",
"luggage.csv",
"seat_assignments.csv",
]

def extract_create_statements(path):
    with open(path, "r") as f:
        content = f.read()
    idx = content.find("INSERT INTO")
    if idx != -1:
        content = content[:idx]
    stmts = []
    for m in re.finditer(r"CREATE TABLE.*?;", content, re.DOTALL):
        stmts.append(m.group())
    return stmts

def load_csv(cursor, csv_file, table_name, col_map):
    path = os.path.join(DATA_DIR, csv_file)
    with open(path, "r") as f:
        reader = csv.reader(f)
        headers = next(reader)
        columns = [col_map.get(h, h) for h in headers]
        placeholders = ", ".join(["%s"] * len(columns))
        col_str = ", ".join(columns)
        sql = f"INSERT IGNORE INTO {table_name} ({col_str}) VALUES ({placeholders})"
        rows = []
        for row in reader:
            cleaned = []
            for val in row:
                if val.strip() in ("NULL", ""):
                    cleaned.append(None)
                else:
                    cleaned.append(val.strip())
            rows.append(cleaned)
        if rows:
            cursor.executemany(sql, rows)

def main():
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()
    cursor.execute(f"DROP DATABASE IF EXISTS {DATABASE}")
    cursor.execute(f"CREATE DATABASE {DATABASE}")

```

```

cursor.execute(f"USE {DATABASE}")

for stmt in extract_create_statements(SQL_FILE):
    cursor.execute(stmt)
print("Schema created.")

cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
for csv_file in LOAD_ORDER:
    table = TABLE_MAP.get(csv_file, csv_file.replace(".csv", ""))
    col_map = COLUMN_MAP.get(csv_file, {})
    load_csv(cursor, csv_file, table, col_map)
    conn.commit()
    print(f"  Loaded {csv_file} -> {table}")
cursor.execute("SET FOREIGN_KEY_CHECKS = 1")

print("\nRow counts:")
for csv_file in LOAD_ORDER:
    table = TABLE_MAP.get(csv_file, csv_file.replace(".csv", ""))
    cursor.execute(f"SELECT COUNT(*) FROM {table}")
    count = cursor.fetchone()[0]
    print(f"  {table}: {count}")

cursor.close()
conn.close()

if __name__ == "__main__":
    main()

```

Testing: The following seven queries test the database and generate meaningful reports for a business user. Every query spans multiple tables. Five use grouping functions (Queries 1, 2, 3, 4, and 7) and two use subqueries (Queries 5 and 6), which exceeds the requirement of at least three grouping queries and two subqueries. The output of each query is shown directly beneath it.

Query One:

```

SELECT orig.airport_code AS origin, dest.airport_code AS destination,
       SUM(p.amount_paid) AS total_revenue
FROM payment p JOIN booking b JOIN booking_flight bf
      JOIN flight f JOIN route r JOIN airport orig JOIN airport dest
ON p.booking_id = b.booking_id AND b.booking_id = bf.booking_id
   AND bf.flight_id = f.flight_id AND f.route_id = r.route_id
   AND r.origin_airport_id = orig.airport_id
   AND r.destination_airport_id = dest.airport_id
WHERE p.payment_status = 'Paid'
GROUP BY orig.airport_code, dest.airport_code
ORDER BY total_revenue DESC;

```

Output:

origin	destination	total_revenue
JFK	HNL	33727.42
LAX	MIA	21349.94
SFO	ORD	20873.20
ATL	MIA	19549.31
LAX	SEA	18846.26
LAX	JFK	16500.91
SEA	DEN	16134.02
JFK	MCO	15411.70
SJC	PDX	14194.37
AUS	MCO	13611.64
SFO	DFW	13567.37
LAX	ORD	13457.95
PDX	HNL	12817.49
BOS	MCO	12744.17
SFO	DEN	12411.46
ORD	ATL	10739.48
SFO	SAN	10244.95
DEN	MSP	9945.66
ORD	MSP	9885.99
LAX	LAS	9809.27
ORD	DFW	8641.53
ORD	DEN	8450.71
DEN	SLC	8241.97
MSP	SLC	7631.14
LAX	DEN	7594.12

Query Two:


```

SELECT p.first_name, p.last_name, ep.employee_number,
       a.airline_name, COUNT(f.flight_id) AS flights_flown
FROM employee_passenger ep
     JOIN passenger p
     JOIN airline a
     JOIN flight f
ON ep.passenger_id = p.passenger_id
   AND ep.airline_id = a.airline_id
   AND f.airline_id = a.airline_id
WHERE ep.job_title = 'Pilot'
GROUP BY ep.passenger_id, p.first_name,
         p.last_name, ep.employee_number, a.airline_name
ORDER BY flights_flown DESC;

```

Output:

	first_name	last_name	employee_number	airline_name	flights_flown
	Marcus	Webb	EMP-UA-0001	United Airlines	31
	James	Carter	EMP-DL-0001	Delta Air Lines	29
	Brian	Tran	EMP-WN-0001	Southwest Airlines	27

Query Three:

```

SELECT f.flight_id, Count(passenger_id)
FROM booking_flight bf JOIN flight f
ON bf.flight_id = f.flight_id
GROUP BY flight_id
ORDER BY count(passenger_id) DESC
LIMIT 1;

```

Output:

flight_...	Count(passenger_...	
64	10	

Query Four:

```

SELECT f.flight_id, f.departure_time,
AVG(YEAR(CURDATE()) - YEAR(ap.date_of_birth)) as avg_age
FROM flight f
JOIN booking_flight bf ON f.flight_id = bf.flight_id
JOIN passenger p ON bf.passenger_id = p.passenger_id
JOIN adult_passenger ap ON p.passenger_id = ap.passenger_id
GROUP BY f.flight_id, f.departure_time
ORDER BY f.flight_id
LIMIT 10;

```

Output:

flight_...	departure_time	avg_age
1	2026-06-01 12:15:00	37.4000
2	2026-06-01 18:00:00	38.1667
3	2026-06-01 12:45:00	41.0000
4	2026-06-01 10:45:00	46.3333
5	2026-06-01 08:00:00	36.3333
6	2026-06-02 08:45:00	45.5000
7	2026-06-02 12:00:00	45.8000
8	2026-06-02 08:15:00	40.0000
9	2026-06-02 16:15:00	47.7500
10	2026-06-02 08:15:00	39.0000

Query Five:

```

SELECT passenger_id, first_name, last_name
FROM passenger
WHERE passenger_id IN(SELECT passenger_id
FROM booking_flight bf JOIN seat_assignment sa
ON bf.booking_flight_id = sa.booking_flight_id
WHERE seat_number = "14E")
ORDER BY passenger_id;

```

Output:

passenger_id	first_name	last_name
1	Alexander	Bailey
2	Ronald	Cox
3	Barbara	Jones
4	Megan	Gutierrez
5	Emily	Adams
9	Michael	Kim
12	Edward	Williams

Query Six:

```

SELECT p.passenger_id, p.first_name, p.last_name
FROM passenger p JOIN booking_flight bf
ON p.passenger_id = bf.passenger_id
WHERE flight_id = "27"
ORDER BY last_name ASC, first_name ASC;

```

Output:

passenger_id	first_name	last_name
11	Heather	Jackson
3	Barbara	Jones
7	Camila	Price

Query Seven:

```

SELECT f.flight_id, AVG(bf.ticket_price)
FROM flight f JOIN booking_flight bf
ON f.flight_id = bf.flight_id
GROUP BY flight_id
HAVING AVG(ticket_price) > (SELECT AVG(ticket_price)
FROM booking_flight)
ORDER BY AVG(bf.ticket_price)
LIMIT 10;

```

Output:

flight_...	AVG(bf.ticket_pri...
53	236.180000
163	236.801250
32	238.853333
245	238.863333
168	238.998000
192	239.253333
179	239.946667
216	241.275000
15	241.402500
97	241.443333